DEVELOPER WEBINAR · SESSION 03

Agent Registry on MOI

Register an on-chain agent

Give an agent a verifiable identity on devnet, end to end — owner, wallet, and discoverable profile, written to the chain in one SDK call.

Adithya · Ecosystem, Sarva Labs

Session 3 · MOI Builders Series

Agents are starting to call agents

Sessions 1 & 2 gave MOI participants their own identity and their own assets. The missing piece for an agentic economy: agents need identity too — a verifiable answer to who an agent is, who owns it, and where it runs.

01

No shared identity

An agent is just an endpoint. Nothing on-chain says who it is or who stands behind it.

02

No discovery

You can't find agents, filter by skill, or verify an operator without a registry to query.

03

No trust boundary

The moment agent A delegates to agent B, identity is the floor everything else builds on.

Today: we register an agent's identity on-chain — the substrate authority gets enforced on later.

The session, end to end

01 **What the registry is**

On-chain profile vs off-chain card. Where identity actually lives.

02 **The SDK surface**

js-moi-agent-registry — one class, write + read methods.

03 **Connect to devnet**

Wallet + VoyageProvider. Ping the contract before anything else.

04 **Register an agent**

Live demo: createAgent — build card, upload, register in one call.

05 **Verify on-chain**

getAgentProfile — owner, wallet, status, timestamps. This is provenance.

06 **Where it's going**

Identity today; delegated authority + cascade revocation next.

07 **Bounty — ship & earn**

Register your own agent, submit the agent_id. Verified = Builder Points.

Lean on-chain, full card off-chain

On most systems an agent is just a URL with no provenance. On MOI, registration writes a lean, verifiable profile to the chain and points at a richer agent card you host. The chain stores the pointer, not the payload.

◆ ON-CHAIN — AgentRegistryLogic

owner	who controls it
agent_wallet	where it operates
url	service endpoint
card_uri	pointer to the card →
status	ACTIVE / PAUSED / DEPRECATED
created_at · updated_at	provenance

○ OFF-CHAIN — the agent card (URI you control)

- name · description · version
- capabilities — streaming, push…
- skills[] — id, name, tags
- preferred_transport
- input / output modes
- protocol-agnostic: a2a · uagent · mcp · custom

Why it matters: anyone can ask the chain who owns an agent — without trusting the agent's own word.

One class. The whole API.

AgentRegistry wraps the registry logic on MOI. Writes change on-chain state and cost fuel; reads are free queries. The contract's logic ID and manifest are baked in — you bring a wallet and an uploader.

WRITE

createAgent build card + upload + register

registerAgent register a card_uri you already hold

updateCard change url / card_uri · owner-only

setAgentStatus ACTIVE / PAUSED / DEPRECATED

transferAgent hand ownership to another wallet

READ

getAgentProfile full on-chain record + found flag

getMyAgents ids owned by your wallet

getAgentsByOwner ids owned by any wallet

getAllAgentIds every registered agent

getAgentCount total registered (bigint)

Peer dep: js-moi-sdk ≥ 0.7.0-rc15 · install both packages together.

Install, connect, ping the contract

Two packages: the SDK is a peer dependency of the registry. Connect a devnet wallet, then read getAgentCount() as a liveness check before any writes.

connect.ts

```
# two packages — the SDK is a peer dependency
npm install js-moi-sdk js-moi-agent-registry

import { Wallet, VoyageProvider } from 'js-moi-sdk'
import { AgentRegistry } from 'js-moi-agent-registry'

const provider = new VoyageProvider('devnet')
const wallet   = await Wallet.fromMnemonic(mnemonic, path)
wallet.connect(provider)

# liveness check — reach the contract before writing
```

Check: if getAgentCount() returns a number, you're connected and ready to register.

The uploader is yours to provide

AgentRegistry.init takes your wallet and an uploader: a function that receives the card JSON and returns a resolvable URI. Today we hand you a hosted uploader — no IPFS keys, real URLs, genuine provenance.

registry.ts

```ts
const registry = await AgentRegistry.init({
  wallet,
  // card JSON in → resolvable URI out
  uploader: async (cardJson) => {
    const res = await fetch(UPLOAD_URL, {
      method: 'POST', body: cardJson,
    })
    const { uri } = await res.json()
    return uri  // https://…  or  ipfs://…
  },
})
```

Baked into the SDK

Logic ID + manifest ship inside the package. You only bring a wallet and an uploader.

TODAY'S DEMO

Hosted uploader — you just call it. No Pinata, no keys. Your card lands at a real URL.

Heads up: no uploader in init() → createAgent throws UploaderRequiredError.

One call: built, uploaded, on-chain

createAgent takes a protocol descriptor and your agent details. It builds the card, runs your uploader, and writes the profile — returning the new agent_id.

register.ts

```ts
const agentId = await registry.createAgent(
  { protocol: 'a2a', protocolVersion: '1.0' },
  {
    name:        'Weather Agent',
    description: 'Real-time weather forecasts.',
    version:     '1.0.0',
    url:         'https://weather-agent.example.com',
    agentWallet: '0xabc...',
    skills: [{ id: 'get-forecast', name: 'Get Forecast' }],
  }
)
console.log(agentId)  // → "agent_1"
```

Live demo: run npm run register — agent_id printed in seconds. We'll verify it next on Voyage.

The chain is now the source of truth

Read the profile straight back. Owner, wallet, card URI, status, and timestamps all come from the chain — no screenshots, no trust-me. getAgentProfile never throws on a missing id; it returns found: false.

verify.ts

```
const { profile, found } =
  await registry.getAgentProfile(agentId)

profile.owner        // you
profile.agent_wallet // where it runs
profile.card_uri     // your hosted card
profile.status       // "ACTIVE"
profile.created_at   // bigint · genesis
profile.updated_at   // bigint · last change
```

This read is provenance

Anyone can ask the chain who owns this agent and when it came into existence.

Change the card and updated_at moves — while created_at and owner hold.

Identity now. Authority next.

The agent is a living record: pause it, update its card, transfer it — all owner-gated, all on-chain. But identity alone can't bound a delegate. That's where MOI is headed.

setAgentStatus

Pause / resume — flip ACTIVE ↔ PAUSED. Every change timestamped.

updateCard

Ship a new version or endpoint. The id stays; the pointer moves.

transferAgent

Move ownership to a new wallet — one atomic op, no accept step.

WHERE THIS IS GOING

Trust transitivity: you vetted agent #1. It delegates to #2 you never saw. Identity can't bound what #2 may do.

DAT — delegated authority tokens scope what a delegate may do.

Cascade revocation — pull #1's authority and everything downstream dies with it.

The throughline: you can't delegate authority to an agent that has no on-chain identity. Today we built that identity.

Ship what you just learned.

Register your own agent on devnet and submit the agent_id. Verified submissions earn Builder Points — redeemable for KMOI when mainnet launches.

01

Run the demo

Use today's scripts. Register your own agent on devnet with the hosted uploader.

02

Grab the agent_id

Copy the agent_id printed in your terminal. That's your proof of work.

03

Submit the form

Handle, wallet, agent_id, txn hash. Bonus: 2+ skills and a PAUSE→ACTIVE round-trip.

FORM → **[Google Form link — drop in chat & pin in Telegram]**

Deadline: [set ~Sunday 11:59 PM EST] · Verified on-chain via getAgentProfile · winners announced after the session

Four things that'll trip you live

None are hard once you know them — but each has stopped a live demo cold. Pre-flight these before the session.

version

js-moi-sdk is on a release candidate (≥0.7.0-rc15). A cached or mismatched version throws confusing errors — use the pinned repo, not a fresh install.

uploader

No uploader in init() → UploaderRequiredError on createAgent. We hand you a hosted one today, so just call it.

owner-only

updateCard, setAgentStatus, transferAgent all reject from a non-owner wallet. Register and manage from the same wallet.

typed errors

TransactionError, QueryError, UploaderError — catch by type. They tell you exactly which stage failed.

Rule of thumb: reads throw QueryError · writes throw TransactionError · uploads throw UploaderError.

SESSION 03 · RECAP

From zero to a verifiable on-chain agent.

01

Registry = identity

Lean profile on-chain, full card off-chain. Owner, wallet, status, timestamps.

02

One call to register

createAgent builds the card, uploads it, and writes the profile — returns agent_id.

03

Provenance is a read

getAgentProfile makes the chain the source of truth. Authority comes next.

Questions in the chat · next session: delegated authority & DAT